

CT520 – Algoritmos e Estruturas de Dados

Curso EN-AEL – 5º Ano

2º Semestre

Docente: Prof. Cortez e Correia

cortez.correia@marinha.pt

Complexidade algorítmica

Objetivos de aprendizagem

- Fornecer conhecimentos para compreender, relativamente aos algoritmos:
 - Eficiência temporal
 - Espaço ocupado

Sumário

- Complexidade algorítmica
 1. Temporal
 2. Espacial

Complexidade algorítmica

- Algoritmo
 - procedimento computacional bem definido que recebe um conjunto de valores, como entrada e produz um conjunto de valores, como saída
 - uma sequência de etapas computacionais que transformam dados de entrada em dados de saída.
 - **Propriedades:**
 - Não ambíguo
 - Finito
 - Termina
 - Preciso

Complexidade algorítmica

- Complexidade Espacial
 - quantidade memória que um algoritmo necessita para executar
- Complexidade Temporal
 - quantidade de tempo um algoritmo necessita para executar
- Como medir o tempo de execução?
 - um computador recente será mais rápido, do que um computador antigo (super-computador vs smartphone)
 - **Solução:** considerar o número de instruções executadas critério genérico e independente da máquina em que o algoritmo é executado.

Complexidade algorítmica

- **Eficiência temporal** medida em termos do número de operações elementares executadas:
 - Aritméticas (+, -, *, /), lógicas (relacionais), atribuição → 1 operação
 - Ciclos → n operações
 - Acesso a memória (e.g. variável, *array*), chamada de função → 1 operação

Complexidade algorítmica

- Número de instruções executadas por um algoritmo, em função da dimensão da entrada de dados (n)

```
public int Maximo(int[] v) {  
    int max = v[0];  
    for (int i=0; i<v.Length; i++) {  
        if (v[i]>max) max = v[i];  
    }  
    return max;  
}
```

1

2

$1 + (n + 1) + (2n)$

$(2n) + (2n)$

1

$7n + 6$

Complexidade algorítmica

- Peso da componente de maior expoente

k	n=10*k	$20n^3+32*n^2+60*n+1000$	$20n^3$		$32*n^2$		$60*n$		1000	
1	10	24800	20000	81%	3200	13%	600	2%	1000	4%
2	20	175000	2E+05	91%	12800	7%	1200	1%	1000	1%
4	40	1334600	1E+06	96%	51200	4%	2400	0%	1000	0%
8	80	10450600	1E+07	98%	2E+05	2%	4800	0%	1000	0%
16	160	82749800	8E+07	99%	8E+05	1%	9600	0%	1000	0%
32	320	658657000	7E+08	99%	3E+06	0%	19200	0%	1000	0%
64	640	5256026600	5E+09	100%	1E+07	0%	38400	0%	1000	0%
128	1280	41995546600	4E+10	100%	5E+07	0%	76800	0%	1000	0%
256	2560	3.35754E+11	3E+11	100%	2E+08	0%	153600	0%	1000	0%
512	5120	2.68519E+12	3E+12	100%	8E+08	0%	307200	0%	1000	0%
1024	10240	2.14782E+13	2E+13	100%	3E+09	0%	614400	0%	1000	0%
2048	20480	1.71812E+14	2E+14	100%	1E+10	0%	1228800	0%	1000	0%

Complexidade algorítmica

- Peso das constantes

k	n=10*k	20n ³	n ³	%
1	10	20000	1000	5%
2	20	2E+05	8000	5%
4	40	1E+06	64000	5%
8	80	1E+07	5E+05	5%
16	160	8E+07	4E+06	5%
32	320	7E+08	3E+07	5%
64	640	5E+09	3E+08	5%
128	1280	4E+10	2E+09	5%
256	2560	3E+11	2E+10	5%
512	5120	3E+12	1E+11	5%
1024	10240	2E+13	1E+12	5%
2048	20480	2E+14	9E+12	5%

tempo (m)	5% (m)
1440	72
60	3
15	0.75
1	0.05

Complexidade algorítmica

- Para analisar algoritmos temos que considerar o **número de itens n** (dimensão do *input*) a ser processado
- **Objetivo:** obter uma estimativa razoável de como o tempo de execução do algoritmo depende do número de n itens a processar (dimensão dos dados a tratar)
 - **Análise assintótica: ordem de crescimento (comportamento assintótico)** como o algoritmo se comporta e escala com o elevado crescimento dos dados de entrada

Complexidade algorítmica

- Exemplo: tempo de execução de 2 algoritmos de ordenação
 - ordenação de 10 itens → 100 ms
 - a) ordenação de 100 itens → 1 000 ms $10n$
 - b) ordenação de 100 itens → 10 000 ms n^2
- Por que é a) é melhor b)? Porque escala melhor!
 - Há que ter a certeza de que as operações sobre os dados não tem um custo demasiado alto na execução total do programa
 - A preferência vai para **algoritmos determinísticos** onde os tempos de execução são aproximadamente **lineares**

Complexidade algorítmica

- Comparação de tempos de execução logarítmica, linear, quadrática e cúbica

log(n)			
k	n=10*k	ln(n)	ln(n)/ln(10)
1	10	3.321928	1.00
2	20	4.321928	1.30
4	40	5.321928	1.60
8	80	6.321928	1.90
16	160	7.321928	2.20
32	320	8.321928	2.51
64	640	9.321928	2.81
128	1280	10.32193	3.11
256	2560	11.32193	3.41
512	5120	12.32193	3.71
1024	10240	13.32193	4.01
2048	20480	14.32193	4.31

$O(\log(n))$

n			
k	n=10*k	n	n/10
1	10	10	1
2	20	20	2
4	40	40	4
8	80	80	8
16	160	160	16
32	320	320	32
64	640	640	64
128	1280	1280	128
256	2560	2560	256
512	5120	5120	512
1024	10240	10240	1 024
2048	20480	20480	2 048

$O(n)$

n^2			
k	n=10*k	n^2	$n^2/100$
1	10	100	1
2	20	400	4
4	40	1600	16
8	80	6400	64
16	160	25600	256
32	320	102400	1 024
64	640	409600	4 096
128	1280	1638400	16 384
256	2560	6553600	65 536
512	5120	26214400	262 144
1024	10240	104857600	1 048 576
2048	20480	419430400	4 194 304

$O(n^2)$

n^3			
k	n=10*k	n^3	$n^3/1000$
1	10	1 000	1
2	20	8 000	8
4	40	64 000	64
8	80	512 000	512
16	160	4 096 000	4 096
32	320	32 768 000	32 768
64	640	262 144 000	262 144
128	1280	2 097 152 000	2 097 152
256	2560	16 777 216 000	16 777 216
512	5120	134 217 728 000	134 217 728
1024	10240	1 073 741 824 000	1 073 741 824
2048	20480	8.58993E+12	8 589 934 592

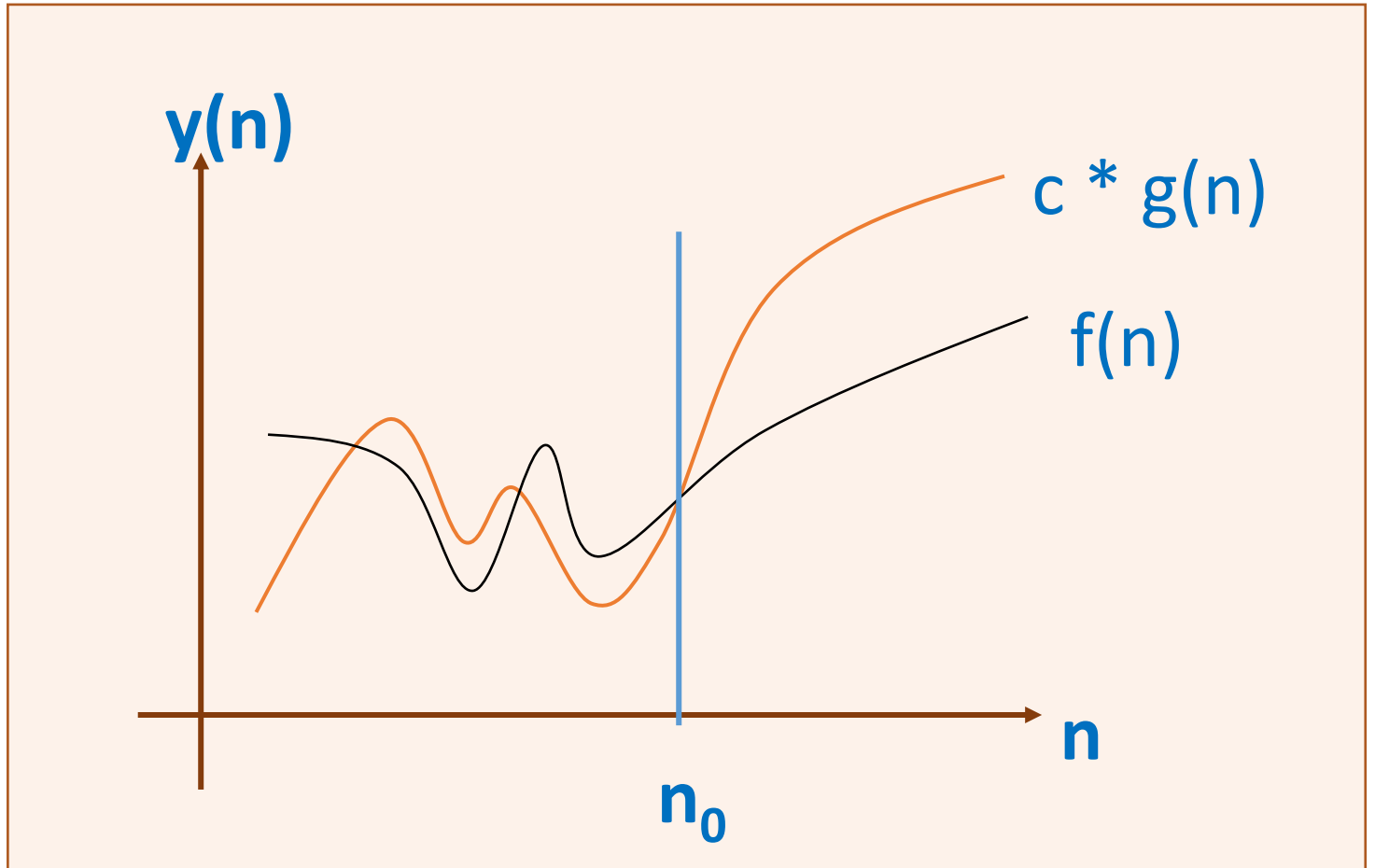
$O(n^3)$

Complexidade algorítmica

- Notação *Big O*
 - Notação Landau
 - Descreve o comportamento majorante de uma função, quando o argumento tende a um determinado valor ou infinito
 - É usado para classificar algoritmos pela forma como respondem (em tempo de processamento ou requisitos de memória) a alterações no tamanho dos dados de entrada
 - Se um algoritmo, em tempo de execução, é de complexidade $O(n^2)$ então também é $O(n^3)$, $O(n^4)$, $O(n^5)$, ...
 - Se $x < 10$ então também é verdade que $x < 100$

Complexidade algorítmica

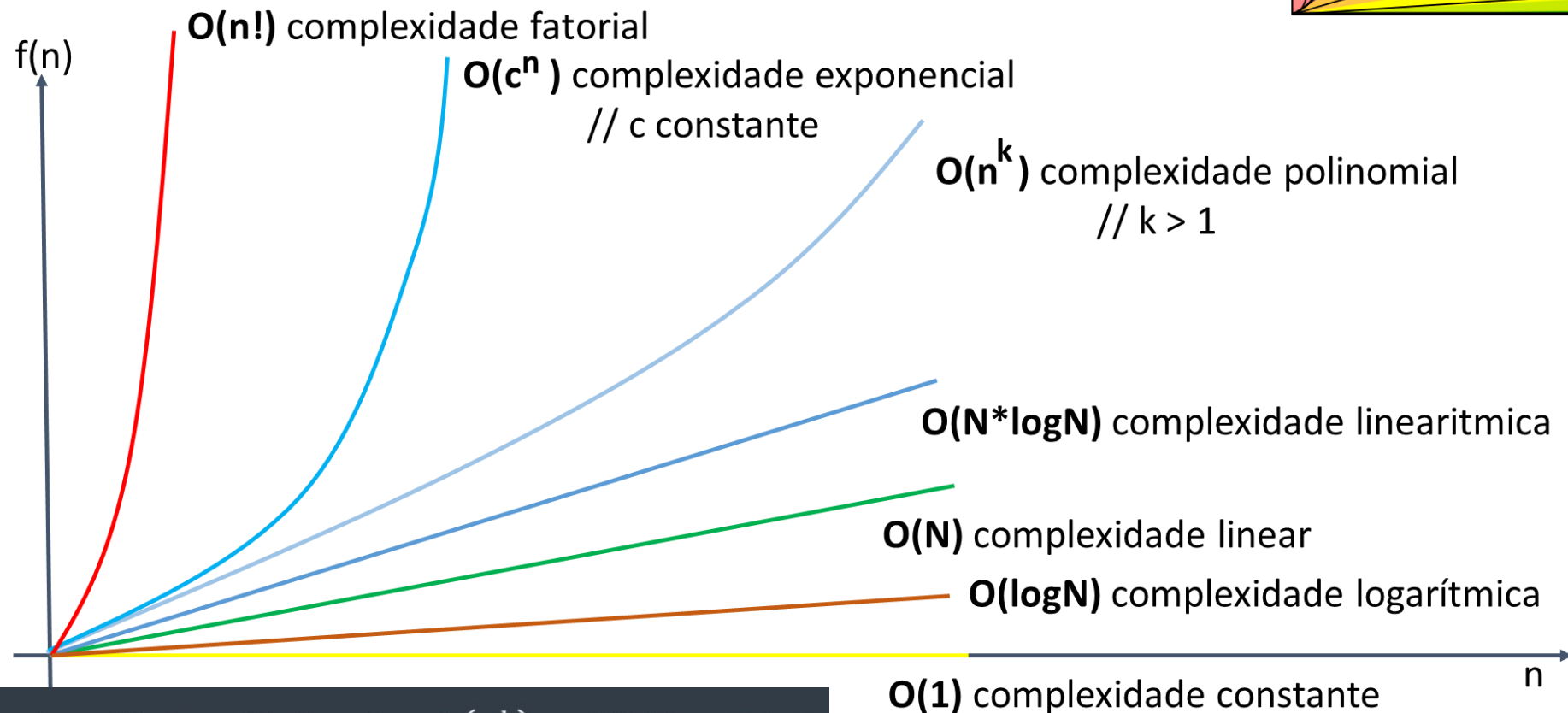
- A expressão $f(n) = O(g(n))$ significa que existe uma constante $c > 0$ e um limiar $n_0 > 0$ tal que, para $n > n_0$ então temos $|f(n)| < c * |g(n)|$
- **Significado:** definição de um limite superior (majorante) para a função $f(n)$



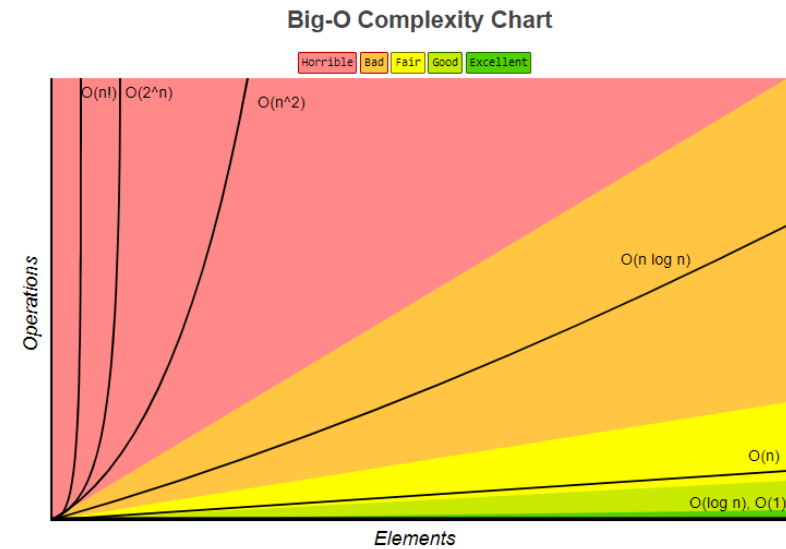
$$f(n) = O(g(n)) \iff \exists c > 0, \exists n_0 > 0 \text{ tal que, se } n > n_0, |f(n)| < c |g(n)|$$

Complexidade algorítmica

- Tempos de execução dos algoritmos



$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^k) < O(k^n) < O(n!)$$



Source:
bigocheatsheet.com

Complexidade algorítmica

$f(n)$	Number of data items processed per:			
	1 minute	1 day	1 year	1 century
n	10	14,400	$5.26 \cdot 10^6$	$5.26 \cdot 10^8$
$n \log n$	10	3,997	883,895	$6.72 \cdot 10^7$
$n^{1.5}$	10	1,275	65,128	$1.40 \cdot 10^6$
n^2	10	379	7,252	72,522
n^3	10	112	807	3,746
2^n	10	20	29	35

Complexidade algorítmica

- Big O (O) $\rightarrow$ pior caso
- Omega (Ω) $\rightarrow$ melhor caso
- Theta (Θ) $\rightarrow$ caso médio (probabilístico)

Complexidade algorítmica

- Álgebra do Big O
 - conjunto de regras e operações matemáticas usadas para descrever o limite superior do tempo de execução ou da complexidade espacial de um algoritmo
1. **Soma de Funções:** $O(f(n)) + O(g(n)) = O(\max(f(n), g(n)))$
 2. **Produto de Funções:** $O(f(n)) \cdot O(g(n)) = O(f(n) \cdot g(n))$
 3. **Transitividade:** Se $f(n) = O(g(n))$ e $g(n) = O(h(n))$, então $f(n) = O(h(n))$.
 4. **Reflexividade:** $f(n) = O(f(n))$
 5. **Simetria:** Se $f(n) = O(g(n))$ e $g(n) = O(f(n))$, desde que $f(n)$ e $g(n)$ cresçam na mesma taxa. (Nota: Esta regra é mais teórica e menos aplicada na prática da análise do Big O, onde o foco está em limites superiores.)
 6. **Encadeamento de Funções (Composição):** Se $f(n) = O(h(n))$ e $g(n) = O(p(n))$, então $f(g(n)) = O(h(p(n)))$.

Complexidade algorítmica

- Recursividade: método da relação de recorrência para cálculo de $T(n)$

```
def fact(n):  
    if n == 1:  
        return 1  
    else:  
        return n * fact(n-1)
```

- Complexidade Temporal
 - $T(n) = O(n)$

$$\begin{aligned}T(1) &= 1 \\T(n) &= T(n-1) + 2 \\T(n) &= T(n-1) + 1 \\T(n) &= (T(n-2) + 1) + 1 \\T(n) &= T(n-2) + 2 \\T(n) &= (T(n-3) + 1) + 2 \\T(n) &= T(n-3) + 3 \\T(n) &= (T(n-4) + 1) + 3 \\T(n) &= T(n-4) + 4 \\T(n) &= T(n-k) + k \\T(n) &= T(n - (n-1)) + (n-1) \\T(n) &= T(n-n+1) + n-1 \\T(n) &= T(1) + n-1 \\T(n) &= 1 + n-1 \\T(n) &= n\end{aligned}$$


Complexidade algorítmica

- Método da relação de recorrência para cálculo de $T(n)$
 - Identificar o número de operações no caso de base
 - identificar a relação de recorrência a partir do caso recursivo
 - simplificar o membro da direita, se possível
 - continuar a substituir até que se evidenciar um padrão
 - escrever a expressão geral com k
 - encontrar o valor de k para alcançar o caso de base
 - substituir k por esse valor
 - simplificar
 - determinar a complexidade do tempo

Complexidade algorítmica

- **Complexidade Espacial:**

- métrica que quantifica a quantidade total de memória temporária ou permanente que um algoritmo necessita para sua execução, em função do tamanho da entrada de dados.
- métrica é vital para a avaliação da eficácia e eficiência dos algoritmos, especialmente em contextos onde a memória é um recurso escasso
- Tanto a complexidade espacial como a temporal são fundamentais para a análise completa do desempenho do algoritmo, mas frequentemente, otimizações de uma podem afetar adversamente a outra .

Complexidade algorítmica

- **Eficiência espacial** medida em termos do espaço suplementar necessário para:
 - Variáveis
 - Estruturas de dados
 - Chamada de métodos
 - Alocação de memória
- Definida em termos da notação Big O

Complexidade algorítmica

- **Análise de Caso Pior (Big O):** refere-se à máxima quantidade de memória que um algoritmo pode requerer, considerando o tamanho da entrada mais desfavorável. Essa análise é crucial para garantir que os algoritmos sejam robustos e capazes de lidar com qualquer entrada dentro do domínio previsto sem exceder as limitações de memória.
- **Complexidade Logarítmica ($O(\log n)$):** algoritmos com essa complexidade espacial, como a pesquisa binária numa estrutura de dados ordenada, são particularmente eficientes porque a quantidade de memória necessária cresce muito lentamente em relação ao tamanho da entrada. Ideal para lidar com grandes volumes de dados.
- **Espaço Auxiliar vs. Espaço Total:** O espaço auxiliar refere-se à memória extra consumida pelo algoritmo, excluindo a memória necessária para as entradas. A complexidade espacial geral considera tanto o espaço de entrada quanto o auxiliar. A distinção é relevante para entender o comportamento do algoritmo em diferentes cenários de uso.

Complexidade algorítmica

- **Estratégias de Otimização:** Incluem a reutilização de recursos, a utilização de estruturas de dados in-place (que operam nos dados de entrada sem utilizar espaço extra significativo), e a escolha de algoritmos com menor complexidade espacial quando a memória é um fator limitante.
- **Impacto da Recursão:** pode aumentar significativamente a complexidade espacial devido à acumulação de chamadas na pilha. A conversão de algoritmos recursivos em iterativos é uma estratégia comum para reduzir o consumo de memória.
- **Algoritmos *In-Place* vs. *Not-In-Place*:** Algoritmos in-place modificam os dados de entrada diretamente e requerem uma quantidade mínima de espaço adicional, enquanto os algoritmos not-in-place podem necessitar de espaço adicional proporcional ao tamanho dos dados de entrada.
- **Importância para Sistemas Embutidos e de Alto Desempenho:** Em sistemas com recursos limitados, como dispositivos embutidos, ou em aplicações de alto desempenho, como bases de dados e sistemas de tempo real, a complexidade espacial torna-se um fator crítico que pode determinar a viabilidade de uma solução algorítmica.

Sumário

- Complexidade algorítmica
 1. Temporal
 2. Espacial